



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE 06: Arreglos y Listas
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumno participante:	Chacha Chango Victor Manuel
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Conocer el manejo de arreglos y listas que permiten la manipulación de grandes cantidades de datos.

Específicos:

- Implementar las estructuras de datos ArrayList y LinkedList en Java (y sus equivalentes vector y list en C++) mediante la codificación de ejercicios prácticos.
- Diseñar algoritmos que apliquen operaciones de gestión de datos (Crear, Leer, Actualizar, Eliminar) dentro de listas dinámicas.
- Comparar el rendimiento y las limitaciones funcionales entre los arreglos estáticos tradicionales y las listas dinámicas.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas
- se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC



- - Computador o laptop.
- - Plataforma educativa de la UTA.
- Internet para consultas.
- - Material de escritorio.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas

2.7 Resultados obtenidos

EJERCICIO 1: ARRAYLIST I - NOMBRES DE ESTUDIANTES

Enunciado: Crear un ArrayList con 5 nombres de estudiantes.

Análisis del problema: Se requiere crear una lista dinámica que almacene cadenas de texto. Se insertarán 5 nombres y luego se utilizará un ciclo iterativo que muestre cada elemento en pantalla.

Equivalencias Java y C++ aplicadas:

- **Clase base:** ArrayList (Java) = vector (C++)
- **Insertión:** add() (Java) = push_back() (C++)
- **Lectura:** get(i) (Java) = [i] (C++)

Variables de entrada, salida y proceso:

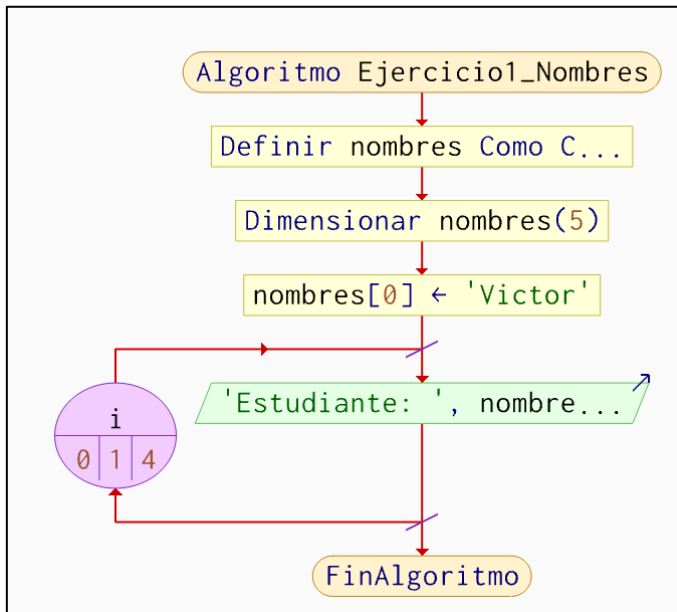
- **Entrada:** 5 nombres (Cadena).
- **Proceso:** Instanciar lista, agregar elementos secuencialmente.
- **Salida:** Impresión de los 5 nombres registrados.

Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio1_Nombres
  Definir nombres Como Caracter
  Dimension nombres[5]
  nombres[0] <- "Victor"
  // ... se repite hasta [4]
  Para i <- 0 Hasta 4 Con Paso 1 Hacer
    Escribir "Estudiante: ", nombres[i]
  FinPara
FinAlgoritmo
```



Diagrama de Flujo



Código en C++:

C++

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<string> estudiantes;
    estudiantes.push_back("Victor");
    estudiantes.push_back("Maria");
    estudiantes.push_back("Luis");
    estudiantes.push_back("Ana");
    estudiantes.push_back("Pedro");
    for(int i = 0; i < estudiantes.size(); i++) {
        cout << "Estudiante " << i+1 << ": " << estudiantes[i] << "\n";
    }
    return 0;
}
```

Código en Java:

Java

```
import java.util.ArrayList;
public class Ejercicio1 {
    public static void main(String[] args) {
        ArrayList<String> estudiantes = new ArrayList<>();
        estudiantes.add("Victor");
        estudiantes.add("Maria");
        estudiantes.add("Luis");
        estudiantes.add("Ana");
        estudiantes.add("Pedro");
        for(int i = 0; i < estudiantes.size(); i++) {
            System.out.println("Estudiante " + (i+1) + ": " +
estudiantes.get(i));
        }
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Inserción	Ejecución inicial	add() / push_back() anexa 5 strings al final	Tamaño de la lista = 5
2. Lectura i=0	Ciclo inicia	get(0) o [0]	Muestra: "Victor"
3. Lectura i=4	Fin del ciclo	get(4) o [4]	Muestra: "Pedro"



EJERCICIO 2: ARRAYLIST I - NOTAS

Enunciado: Crear un ArrayList con 6 notas.

Análisis del problema: Utilizar una lista dinámica que maneje números de punto flotante. Se agregarán 6 notas y se recorrerá la lista para sumar sus valores y calcular el promedio.

Equivalencias Java y C++ aplicadas:

- **Clase base:** ArrayList (Java) = vector (C++)
- **Inserción:** add() (Java) = push_back() (C++)
-

Variables de entrada, salida y proceso:

- **Entrada:** 6 valores decimales.
- **Proceso:** suma = suma + nota, promedio = suma / tamaño.
- **Salida:** Promedio final.

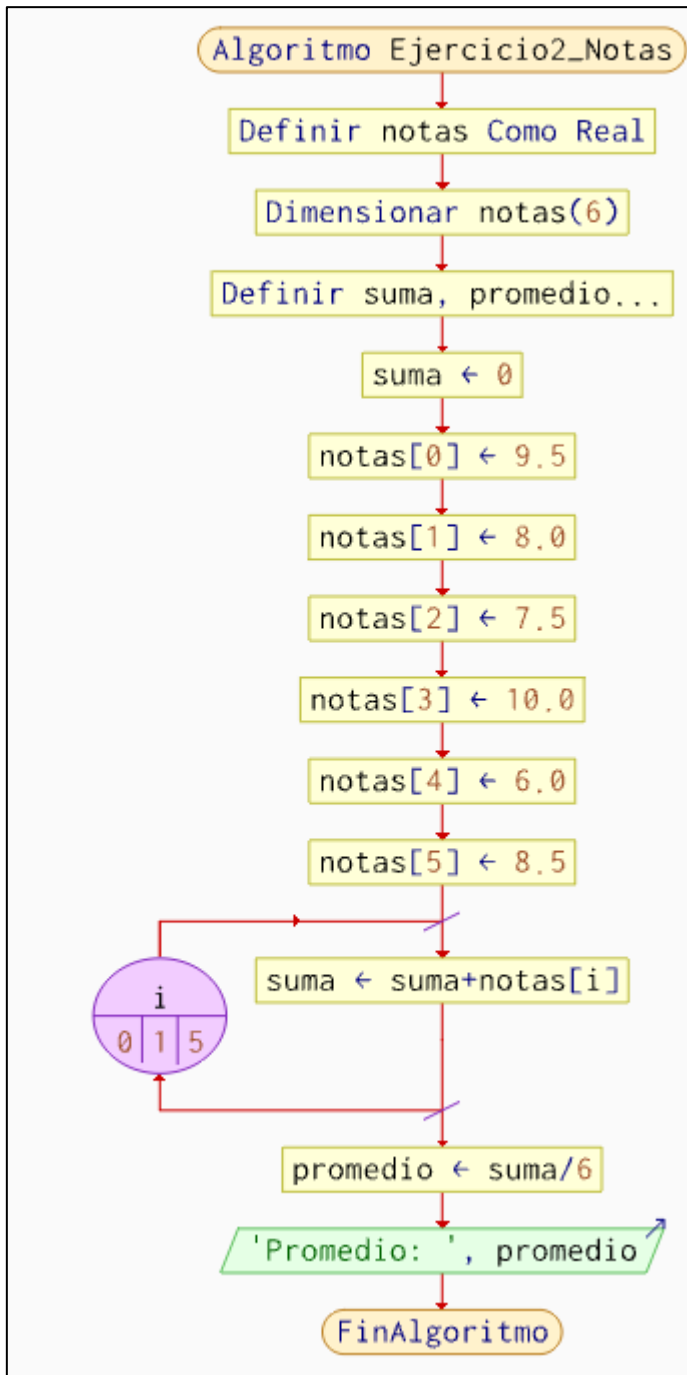
Algoritmo y Pseudocódigo (PSeInt):

Algoritmo Ejercicio2_Notas

```
Definir notas Como Real; Dimension notas[6]
Definir suma, promedio Como Real; suma <- 0
notas[0] <- 9.5; notas[1] <- 8.0; notas[2] <- 7.5
notas[3] <- 10.0; notas[4] <- 6.0; notas[5] <- 8.5
Para i <- 0 Hasta 5 Con Paso 1 Hacer
    suma <- suma + notas[i]
FinPara
promedio <- suma / 6
Escribir "Promedio: ", promedio
```

FinAlgoritmo

Diagrama de Flujo



Código en C++:

C++

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<double> notas = {9.5, 8.0, 7.5, 10.0, 6.0, 8.5}; // usando
initializer list
    double suma = 0;
    for(double n : notas) suma += n;
    cout << "Promedio total: " << (suma / notas.size()) << "\n";
    return 0;
}
```



Código en Java:

Java

```
import java.util.ArrayList;
public class Ejercicio2 {
    public static void main(String[] args) {
        ArrayList<Double> notas = new ArrayList<>();
        notas.add(9.5); notas.add(8.0); notas.add(7.5);
        notas.add(10.0); notas.add(6.0); notas.add(8.5);
        double suma = 0;
        for(Double n : notas) suma += n;
        System.out.println("Promedio total: " + (suma / notas.size()));
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Creación	Ingreso de 6 dobles	Asignación en memoria	size() = 6
2. Acumulación	Recorrido foreach	$9.5+8.0+7.5+10.0+6.0+8.5$	suma = 49.5
3. Promedio	División final	$49.5 / 6$	Imprime "Promedio total: 8.25"



EJERCICIO 3: ARRAYLIST I - REGISTRO Y BÚSQUEDA

Enunciado: Registrar productos y realizar búsquedas.

Análisis del problema: Crear una lista de productos (String). Luego, solicitar al usuario un texto a buscar para iterar y encontrar coincidencias.

Equivalencias Java y C++ aplicadas:

- **Clase base:** ArrayList (Java) = vector (C++)
- **Inserción:** add() (Java) = push_back() (C++)

Variables de entrada, salida y proceso:

- **Entrada:** productoBuscar (Cadena).
- **Proceso:** Búsqueda secuencial.
- **Salida:** Mensaje de "Encontrado" o "No Encontrado".

Algoritmo y Pseudocódigo (PSeInt):

Algoritmo Ejercicio3_Buscar

Definir productos Como Caracter; Dimensionar productos[3]

Definir buscar Como Caracter; Definir enc Como Logico; enc <- Falso

productos[0] <- "Teclado"; productos[1] <- "Mouse"; productos[2] <- "Monitor"

Escribir "Producto a buscar:"

Leer buscar

Para i <- 0 Hasta 2 Hacer

 Si productos[i] = buscar Entonces enc <- Verdadero FinSi

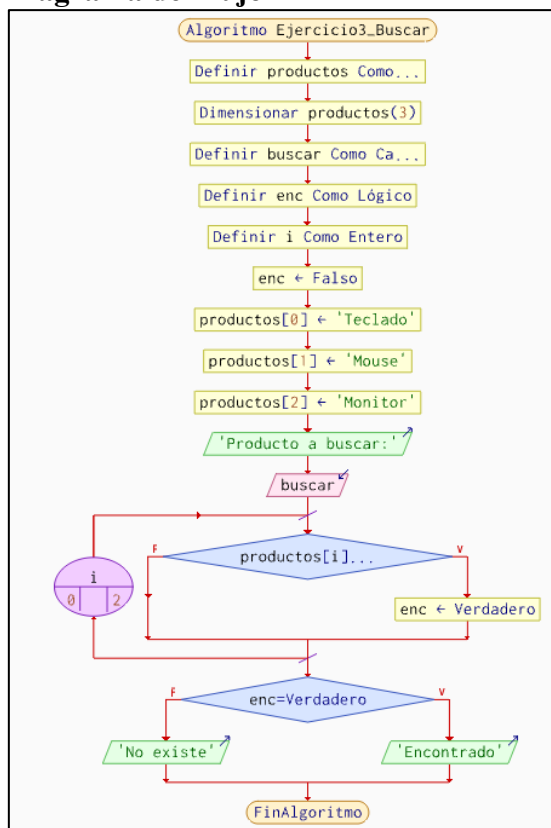
FinPara

Si enc = Verdadero Entonces Escribir "Encontrado" Sino Escribir "No existe"

FinSi

FinAlgoritmo

Diagrama de Flujo





Código en C++:

C++

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<string> prods;
    prods.push_back("Teclado"); prods.push_back("Mouse");
    prods.push_back("Monitor");
    string buscar; cout << "Buscar producto: "; cin >> buscar;
    bool enc = false;
    for(string p : prods) if(p == buscar) enc = true;
    cout << (enc ? "Producto encontrado\n" : "No existe\n");
    return 0;
}
```

Código en Java:

Java

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio3 {
    public static void main(String[] args) {
        ArrayList<String> prods = new ArrayList<>();

        prods.add("Teclado");
        prods.add("Mouse");
        prods.add("Monitor");

        Scanner sc = new Scanner(System.in);

        System.out.print("Buscar producto: ");
        String buscar = sc.next();

        if (prods.contains(buscar)) {
            System.out.println("Producto encontrado");
        } else {
            System.out.println("No existe");
        }

        sc.close();
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Coincidencia	Buscar "Mouse"	La iteración encuentra match	Imprime "Encontrado"
2. Fallo	Buscar "CPU"	Iteración completa no halla match	Imprime "No existe"
3. Sensibilidad	Buscar "teclado"	Falla coincidencia exacta de strings	Imprime "No existe"



EJERCICIO 4: ARRAYLIST II - MENÚ CRUD

Enunciado: Crear un menú CRUD usando ArrayList.

Análisis del problema: Implementar un bucle interactivo (do-while) que gestione la lista.

Permitirá Agregar, Leer, Actualizar y Eliminar.

Equivalencias Java y C++ aplicadas:

- **Clase base:** ArrayList (Java) = vector (C++)
- **Inserción:** add() (Java) = push_back() (C++)
- **Eliminación:** remove() (Java) = erase() (C++)
- **Acceso:** get(i) (Java) = [i] (C++)

Variables de entrada, salida y proceso:

- **Entrada:** opcion (Entero), dato (Cadena), indice (Entero).
- **Proceso:** Mutaciones directas sobre los índices de la lista dinámica.
- **Salida:** Confirmación de operaciones y listado de elementos.

Algoritmo y Pseudocódigo (PSeInt):

Algoritmo Ejercicio4_CRUD

Definir op Como Entero

Repetir

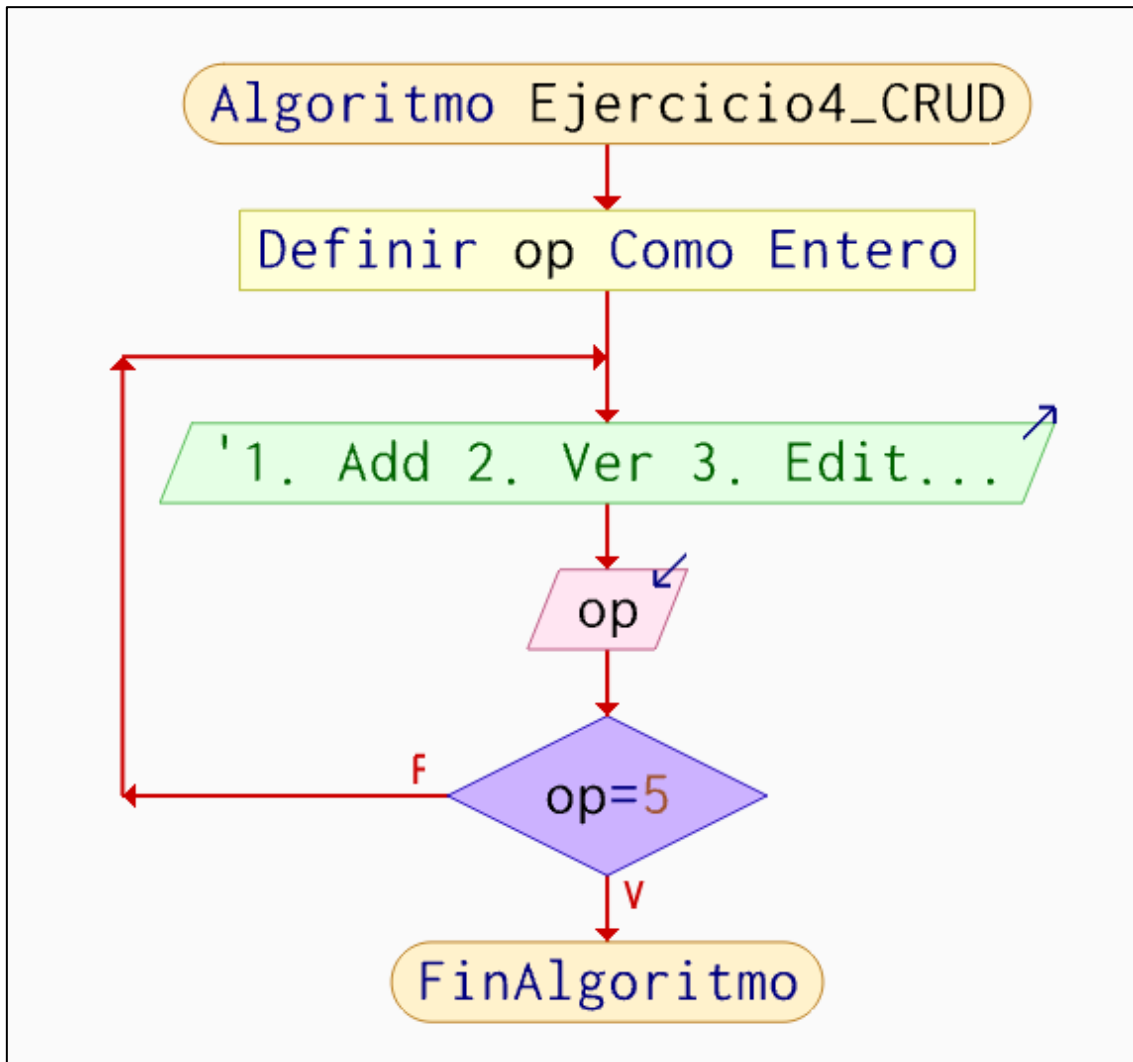
 Escribir "1. Add 2. Ver 3. Editar 4. Borrar 5. Salir"

 Leer op

 Hasta Que op = 5

FinAlgoritmo

Diagrama de Flujo



Código en C++:

C++

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<string> lista; int op, idx; string item;
    do {
        cout << "1.Crear 2.Leer 3.Editar 4.Borrar 5.Salir: "; cin >> op;
        if(op == 1) { cin >> item; lista.push_back(item); }
        else if(op == 2) { for(int i=0; i<lista.size(); i++) cout<<i<<": "<<lista[i]<<"\n"; }
        else if(op == 3) { cin>>idx>>item; if(idx<lista.size()) lista[idx] = item; }
        else if(op == 4) { cin>>idx; if(idx<lista.size()) lista.erase(lista.begin()+idx); }
    } while(op != 5);
    return 0;
}
```

Código en Java:

Java

```
import java.util.ArrayList;
import java.util.Scanner;
```



```
public class Ejercicio4 {
    public static void main(String[] args) {
        ArrayList<String> lista = new ArrayList<>();
        Scanner sc = new Scanner(System.in);
        int op;

        do {
            System.out.print("1.Add 2.Ver 3.Update 4.Delete 5.Salir: ");
            op = sc.nextInt();

            if (op == 1) {
                lista.add(sc.next());
            } else if (op == 2) {
                for (int i = 0; i < lista.size(); i++) {
                    System.out.println(i + ":" + lista.get(i));
                }
            } else if (op == 3) {
                int id = sc.nextInt();
                if (id < lista.size()) {
                    lista.set(id, sc.next());
                }
            } else if (op == 4) {
                int id = sc.nextInt();
                if (id < lista.size()) {
                    lista.remove(id);
                }
            }
        } while (op != 5);

        sc.close();
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Agregar	Op 1, ingresa "Dato1"	add / push back	Agrega al índice 0
2. Actualizar	Op 3, ID 0, "NuevoDato"	set / asignación de índice	Sobrescribe el valor
3. Eliminar	Op 4, ID 0	remove() / erase()	Lista queda vacía



EJERCICIO 5: ARRAYLIST II - NOMBRES Y NOTAS

Enunciado: Registrar nombres y notas.

Análisis del problema: Aplicar Programación Orientada a Objetos creando una clase (Alumno) que encapsule nombre y nota. Luego crear una lista dinámica de objetos de esta clase.

Equivalencias Java y C++ aplicadas:

- **Clase base:** ArrayList (Java) = vector (C++)
- **Inserción:** add() (Java) = push_back() (C++)

Variables de entrada, salida y proceso:

- **Entrada:** Nombre y nota para instanciar objetos.
- **Proceso:** lista.add(new Alumno(n, p)).
- **Salida:** Datos encapsulados mostrados iterando la lista.

Algoritmo y Pseudocódigo (PSeInt):

Algoritmo Ejercicio5_Objeto

Definir nombres Como Caracter; Dimension nombres[2]

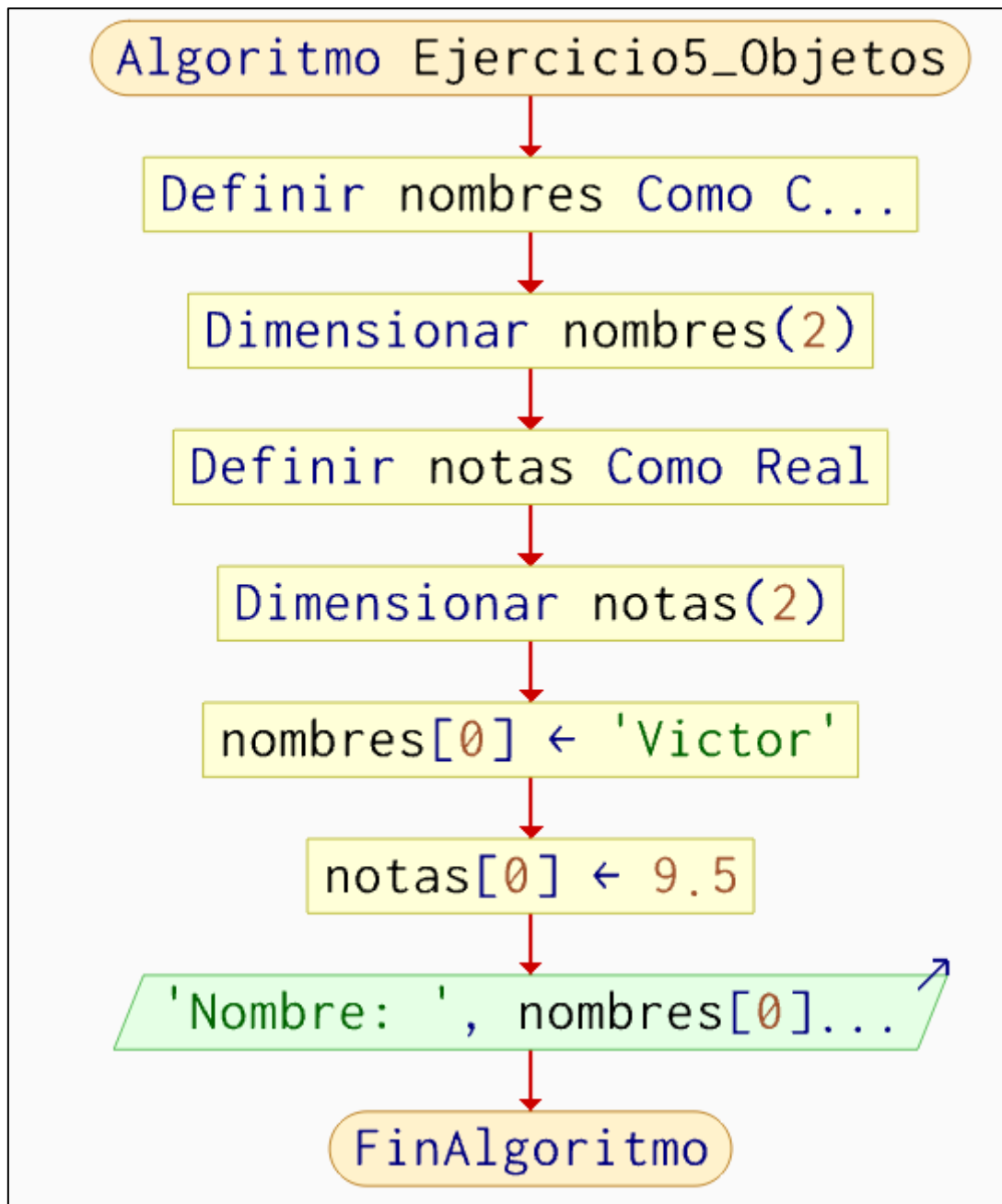
Definir notas Como Real; Dimension notas[2]

nombres[0] <- "Victor"; notas[0] <- 9.5

Escribir "Nombre: ", nombres[0], " - Nota: ", notas[0]

FinAlgoritmo

Diagrama de Flujo



Código en C++:

C++

```
#include <iostream>
#include <vector>
using namespace std;
class Alumno {
public: string nombre; double nota;
    Alumno(string n, double p) : nombre(n), nota(p) {}
};
int main() {
    vector<Alumno> clase;
    clase.push_back(Alumno("Victor", 9.5));
    clase.push_back(Alumno("Maria", 8.0));
    for(Alumno a : clase) cout << a.nombre << " - " << a.nota << "\n";
```



```
return 0;  
}
```

Código en Java:

Java

```
import java.util.ArrayList;  
class Alumno {  
    String nombre; double nota;  
    Alumno(String n, double p) { nombre = n; nota = p; }  
}  
public class Ejercicio5 {  
    public static void main(String[] args) {  
        ArrayList<Alumno> clase = new ArrayList<>();  
        clase.add(new Alumno("Victor", 9.5));  
        clase.add(new Alumno("Maria", 8.0));  
        for(Alumno a : clase) System.out.println(a.nombre + " - " + a.nota);  
    }  
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Instanciación	"Victor", 9.5	Se crea objeto Alumno y va a la lista	1 objeto en memoria
2. Atributos	Acceso a a.nota	get() implícito del atributo	Muestra 9.5
3. Varios objetos	Agregar "Maria", 8.0	add() anexa nuevo objeto compuesto	Tamaño de lista = 2



EJERCICIO 6: ARRAYLIST II - SISTEMA DE INVENTARIO

Enunciado: Crear un sistema de inventario.

Análisis del problema: Se crea la clase Producto (nombre, cantidad, precio). Se almacenan en una lista dinámica, permitiendo calcular el patrimonio total iterando el inventario (cantidad * precio).

Equivalencias Java y C++ aplicadas:

- **Clase base:** ArrayList (Java) = vector (C++)
- **Inserción:** add() (Java) = push_back() (C++)

Variables de entrada, salida y proceso:

- **Proceso:** total = total + (prod.cant * prod.precio).

Algoritmo y Pseudocódigo (PSeInt):

Algoritmo Ejercicio6_Inventario

Definir cant Como Entero; cant <- 10

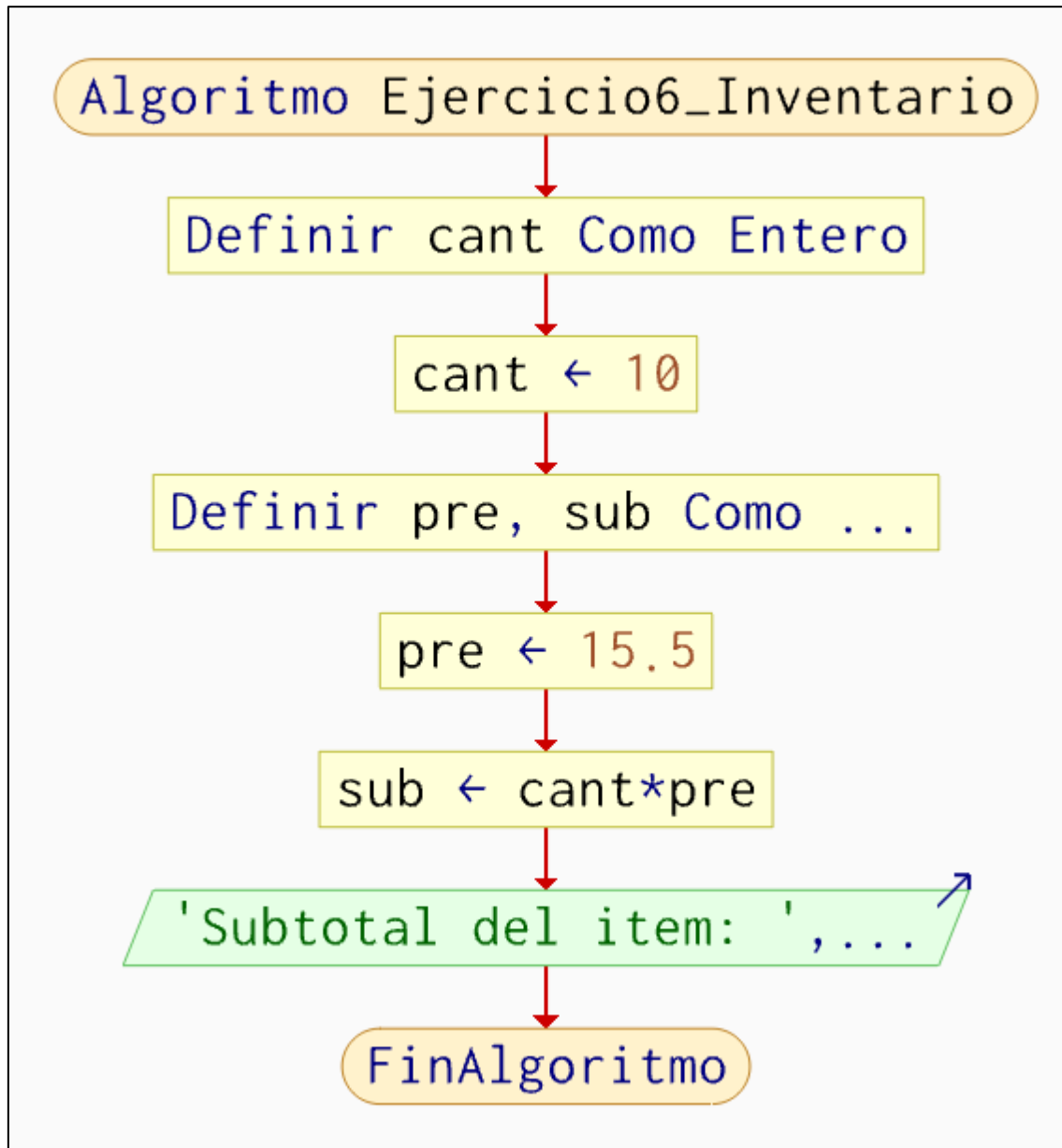
Definir pre, sub Como Real; pre <- 15.5

sub <- cant * pre

Escribir "Subtotal del item: ", sub

FinAlgoritmo

Diagrama de Flujo



Código en C++:

C++

```
#include <iostream>
#include <vector>
using namespace std;
class Prod { public: string nom; int cant; double pre;
    Prod(string n, int c, double p) : nom(n), cant(c), pre(p) {}
};
int main() {
    vector<Prod> inv;
    inv.push_back(Prod("Mouse", 10, 15.5));
    double total = 0;
    for(Prod p : inv) total += (p.cant * p.pre);
    cout << "Valor inventario: $" << total << "\n";
    return 0;
}
```



Código en Java:

Java

```
import java.util.ArrayList;
class Prod {
    String nom; int cant; double pre;
    Prod(String n, int c, double p) { nom=n; cant=c; pre=p; }
}
public class Ejercicio6 {
    public static void main(String[] args) {
        ArrayList<Prod> inv = new ArrayList<>();
        inv.add(new Prod("Mouse", 10, 15.5));
        double total = 0;
        for(Prod p : inv) total += (p.cant * p.pre);
        System.out.println("Valor inventario: $" + total);
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Agregar Producto	Mouse, 10, 15.5	instanciación en memoria	Elemento registrado
2. Cálculo Subtotal	Itera primer item	$10 * 15.5\$$	total acumula 155.0
3. Patrimonio Global	Fin del ciclo for	Impresión de la variable total	Muestra "\$155.0"



EJERCICIO 7: ARREGLO VS ARRAYLIST (COMPARACIÓN)

Enunciado: Comparar arreglos y ArrayList.

Análisis del problema: Demostrar en código que un arreglo primitivo tiene un tamaño inmutable (length), mientras que el ArrayList tiene un tamaño variable (size()).

Equivalencias Java y C++ aplicadas:

- **Clase base:** ArrayList (Java) = vector (C++)
- **Inserción:** add() (Java) = push_back() (C++)

Variables de entrada, salida y proceso:

- **Entrada:** Ninguna (datos quemados en código).
- **Proceso:** Instanciación de un arreglo estático con longitud fija (3) y de una lista dinámica. Adición secuencial de elementos a la lista para demostrar su crecimiento.
- **Salida:** Impresión comparativa del tamaño estático (length) vs el tamaño dinámico (size()).

Algoritmo y Pseudocódigo (PSeInt):

Algoritmo Ejercicio7_Comparacion

Definir arr Como Entero

Dimension arr[3]

Definir sizeLista Como Entero

arr[0] <- 1

arr[1] <- 2

arr[2] <- 3

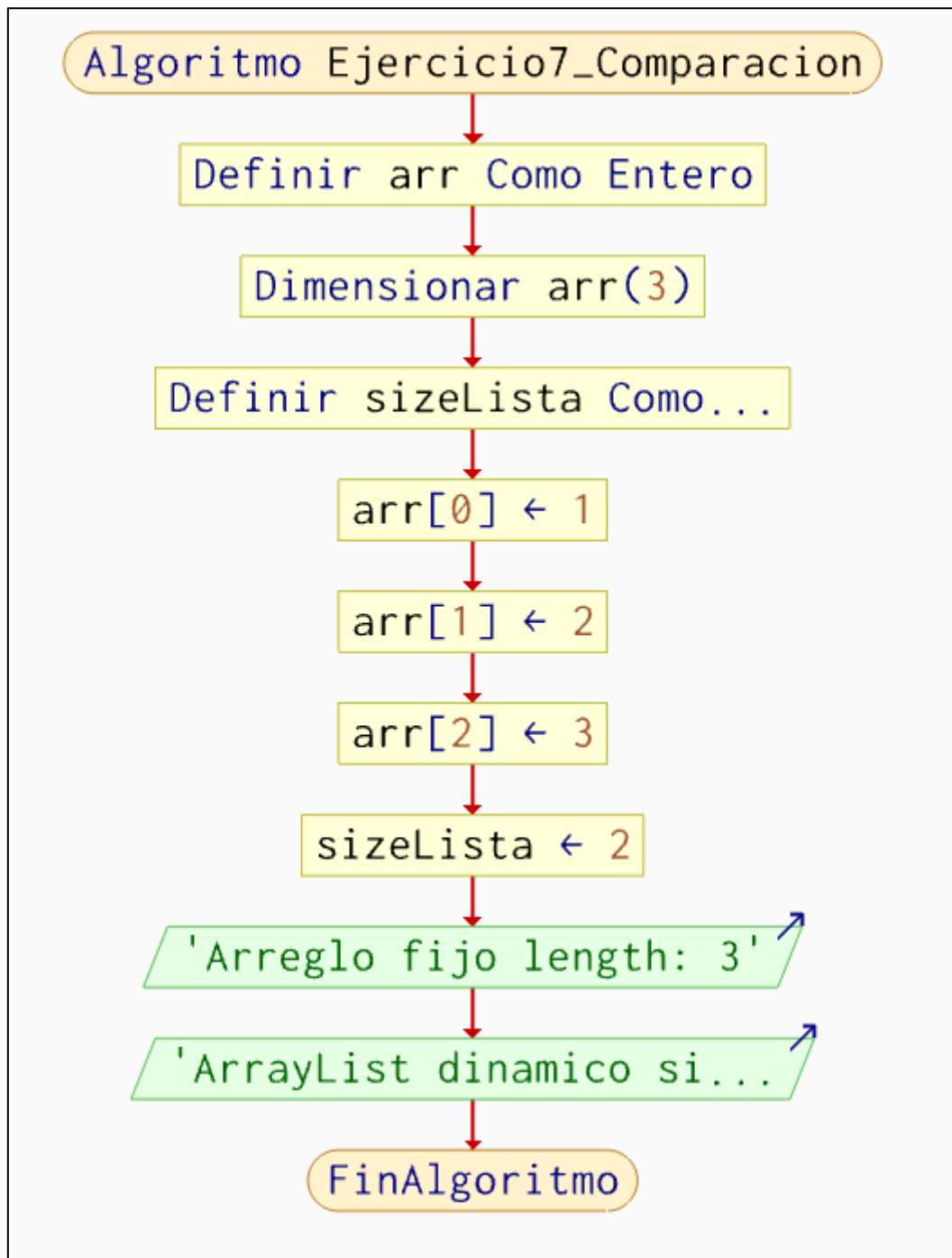
sizeLista <- 2 // Simulando el crecimiento dinámico en ejecución

Escribir "Arreglo fijo length: 3"

Escribir "ArrayList dinamico size: ", sizeLista

FinAlgoritmo

Diagrama de Flujo



Código en C++:

C++

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int arr[3] = {1, 2, 3}; // Estatico, no puede crecer
    vector<int> lista;      // Dinamico
    lista.push_back(1); lista.push_back(2);
    cout << "Size arreglo: 3 | Size vector: " << lista.size() << "\n";
    return 0;
}
```



Código en Java:

Java

```
import java.util.ArrayList;
public class Ejercicio7 {
    public static void main(String[] args) {
        int[] arr = new int[3];
        arr[0] = 1; arr[1] = 2; arr[2] = 3;
        System.out.println("Arreglo fijo length: " + arr.length);

        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(1); lista.add(2);
        System.out.println("ArrayList dinamico size: " + lista.size());
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Tamaño Arreglo	int arr[3]	La memoria reserva 3 espacios inmutables	length muestra 3
2. Tamaño Lista	lista.add(1) y add(2)	El ArrayList crece dinámicamente 2 veces	size() muestra 2
3. Acceso límite	Iterar hasta el límite	Acceder a arr[2] es válido, arr[3] fallaría	Ejecución estable



EJERCICIO 8: ARREGLO VS ARRAYLIST (INSERCIÓN)

Enunciado: Comparar inserción de datos.

Análisis del problema: Forzar una inserción más allá del límite en un arreglo nativo genera una excepción, mientras que hacer lo mismo en una lista dinámica es válido.

Equivalencias Java y C++ aplicadas:

- **Clase base:** ArrayList (Java) = vector (C++)
- **Inserción ilimitada:** add() (Java) = push_back() (C++)

Variables de entrada, salida y proceso:

- **Entrada:** Ninguna.
- **Proceso:** Inserción de 3 elementos en un ArrayList (operación permitida) vs intento de inserción de un tercer elemento en un arreglo limitado a 2 posiciones (operación ilegal). Uso de bloque try-catch para manejar la excepción.
- **Salida:** Mensaje demostrando el éxito de la lista dinámica y mensaje de error capturado del arreglo.

Algoritmo y Pseudocódigo (PSeInt):

Algoritmo Ejercicio8_Insercion

Definir arr Como Entero

Dimension arr[2]

Definir sizeLista Como Entero

sizeLista <- 3 // Simula las 3 inserciones exitosas en la lista

Escribir "ArrayList crece, size: ", sizeLista

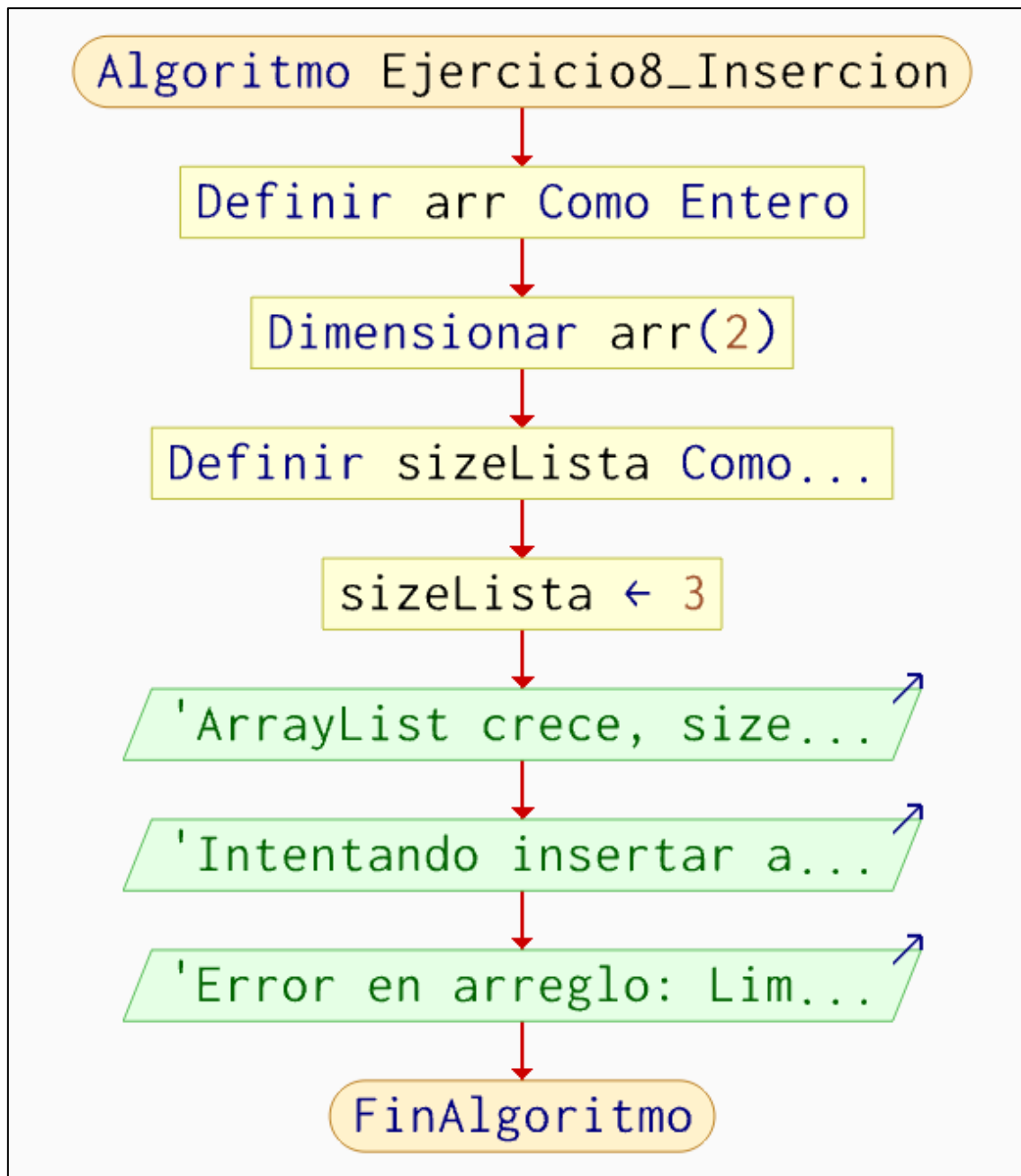
Escribir "Intentando insertar arr[2] <- 30..."

// Simulacion del catch

Escribir "Error en arreglo: Limite superado"

FinAlgoritmo

Diagrama de Flujo



Código en C++:

C++

```
#include <iostream>
```

```
#include <vector>
```

```
#include <stdexcept> // Librería necesaria para atrapar excepciones
```

```
using namespace std;
```

```
int main() {
```

```
    int arr[2]; // Límite estricto de 2 espacios (índices 0 y 1)
```

```
    vector<int> lista;
```

```
    // El vector soporta múltiples inserciones dinámicas sin importar el límite inicial
```

```
    lista.push_back(10);
```



```
lista.push_back(20);
lista.push_back(30); // Funciona perfecto
cout << "Vector crece, size: " << lista.size() << "\n";

// Control de errores para el arreglo
try {
    int indiceAIngresar = 2;

    // Simulamos la validación estricta de límites que hace Java
    if(indiceAIngresar >= 2) {
        throw out_of_range("Limite superado");
    }
    arr[indiceAIngresar] = 30; // Intento de escritura

} catch(const out_of_range& e) {
    cout << "Error en arreglo: " << e.what() << "\n";
}

return 0;
}
```

Código en Java:

Java

```
import java.util.ArrayList;
public class Ejercicio8 {
    public static void main(String[] args) {
        int[] arr = new int[2];
        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(10); lista.add(20); lista.add(30); // Funciona perfecto
        System.out.println("ArrayList crece, size: " + lista.size());
        try {
            arr[2] = 30; // Falla, index maximo es 1
        } catch(Exception e) {
            System.out.println("Error en arreglo: Limite superado");
        }
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Lista dinámica	lista.add(30)	La lista adapta su tamaño en memoria	size actual = 3
2. Arreglo válido	Asignar índice 0 y 1	Ocupa el espacio reservado del array	Sin errores
3. Arreglo desbordado	arr[2] = 30 (posición 3)	Lanza ArrayIndexOutOfBoundsException	Atrapa el error e imprime "Límite superado"



EJERCICIO 9: LINKEDLIST - MANIPULAR TURNOS

Enunciado: Manipular turnos con LinkedList.

Análisis del problema: La clase LinkedList utiliza nodos doblemente enlazados. Son altamente eficientes para operaciones en las puntas (cabeza y cola).

Equivalencias Java y C++ aplicadas:

- **Clase base:** LinkedList (Java) = list (C++)
- **Eliminación (concepto):** remove() (Java) = erase() (C++).

Variables de entrada, salida y proceso:

- **Proceso:** Elimina el nodo cabeza, desplazando el siguiente al inicio.

Algoritmo y Pseudocódigo (PSeInt):

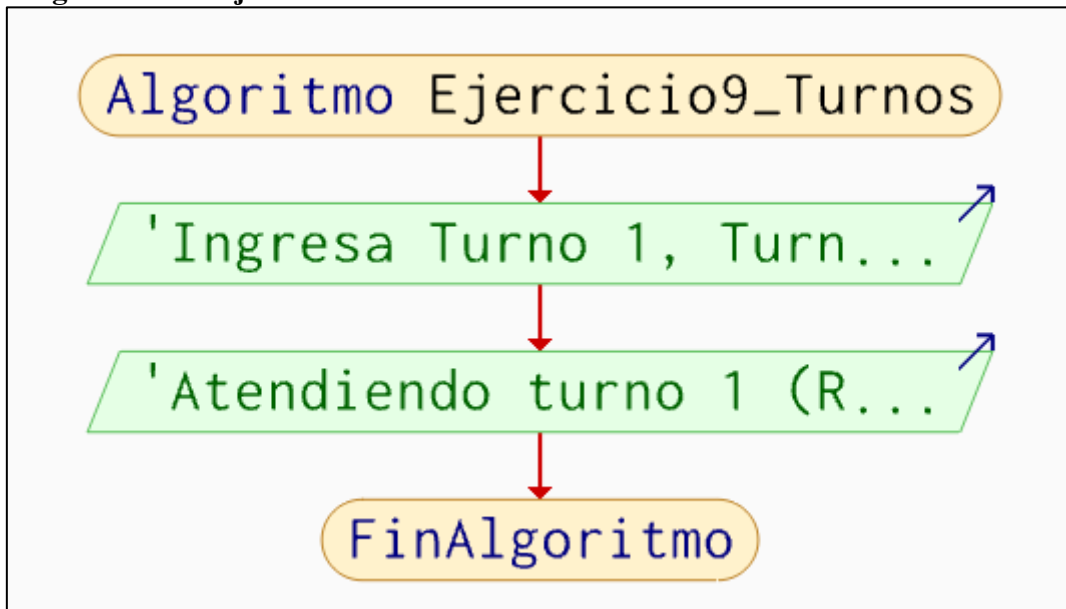
Algoritmo Ejercicio9_Turnos

 Escribir "Ingresa Turno 1, Turno 2..."

 Escribir "Atendiendo turno 1 (Removiendo)..."

FinAlgoritmo

Diagrama de Flujo



Código en C++:

C++

```
#include <iostream>
```

```
#include <list>
```

```
using namespace std;
```

```
int main() {
```

```
    list<string> turnos;
```

```
    turnos.push_back("Turno A"); turnos.push_back("Turno B");
```

```
    cout << "Atendiendo: " << turnos.front() << "\n";
```

```
    turnos.pop_front();
```

```
    cout << "Siguiente: " << turnos.front() << "\n";
```

```
    return 0;
```

```
}
```



Código en Java:

Java

```
import java.util.LinkedList;
public class Ejercicio9 {
    public static void main(String[] args) {
        LinkedList<String> turnos = new LinkedList<>();
        turnos.addLast("Turno A");
        turnos.addLast("Turno B");
        System.out.println("Atendiendo: " + turnos.removeFirst());
        System.out.println("Siguiendo en espera: " + turnos.getFirst());
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Nuevo Turno	Turno A, Turno B	Inserta al final del enlace	Cola con A -> B
2. Atender	Petición de remover inicio	Extrae "A", mueve el Head a "B"	Muestra "Atendiendo: Turno A"
3. Consultar	Petición de leer inicio	Lee el Head sin eliminarlo	Muestra "Turno B"



EJERCICIO 10: LINKEDLIST - FILA DE ATENCIÓN

Enunciado: Simular una fila de atención.

Análisis del problema: Una LinkedList se vaciará utilizando un bucle condicional, que irá despachando a los clientes formados hasta que no quede nadie.

Equivalencias Java y C++ aplicadas:

- **Clase base:** LinkedList (Java) = list (C++)
- **Eliminación (concepto):** remove() (Java) = erase() (C++).

Variables de entrada, salida y proceso:

- **Entrada:** Elementos a encolar.
- **Proceso:** Extracción iterativa hasta que size() == 0.

Algoritmo y Pseudocódigo (PSeInt):

Algoritmo Ejercicio10_Fila

Definir fila Como Entero; fila <- 3

Mientras fila > 0 Hacer

 Escribir "Atendiendo cliente..."

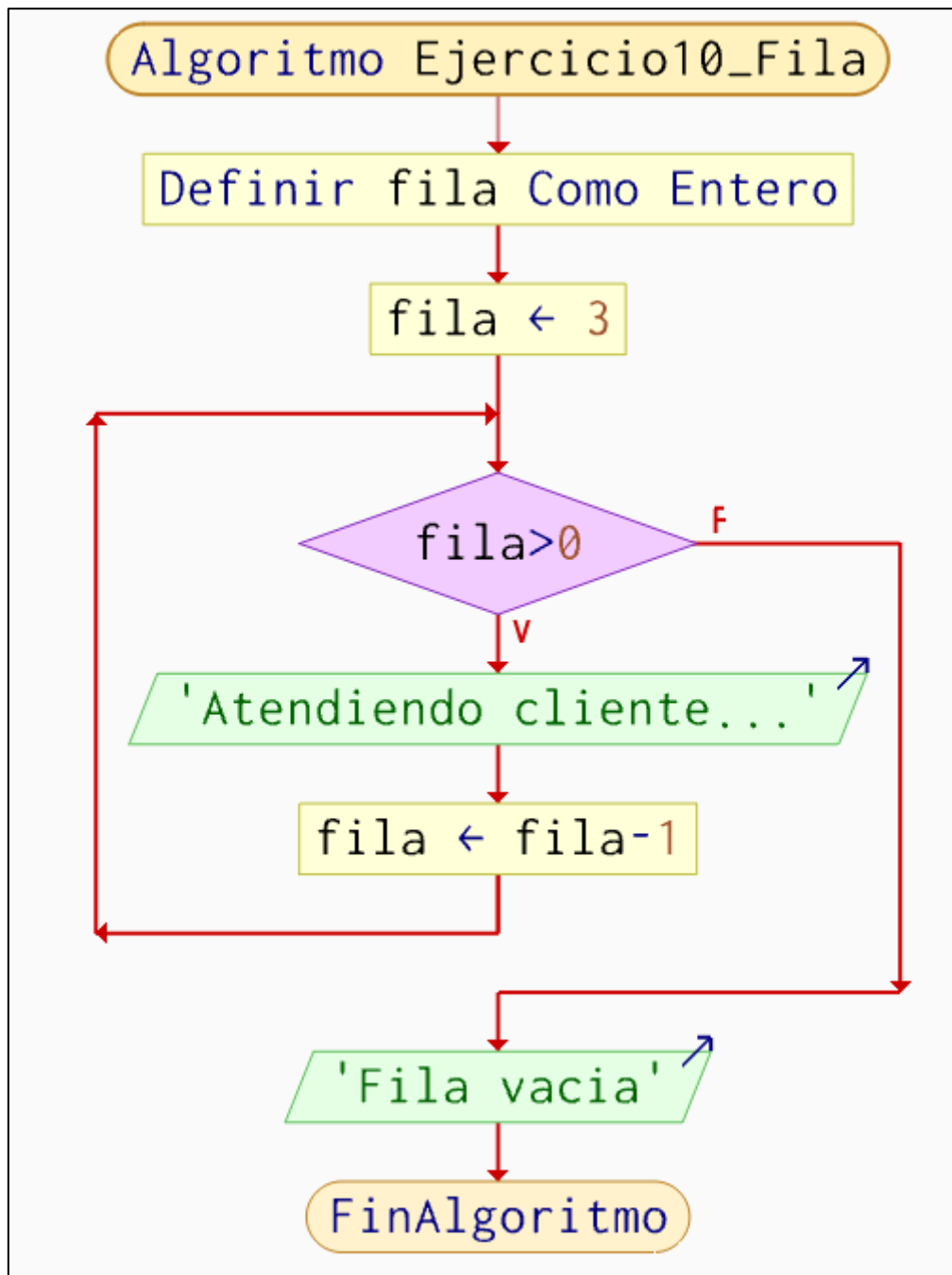
 fila <- fila - 1

FinMientras

Escribir "Fila vacia"

FinAlgoritmo

Diagrama de Flujo



Código en C++:

C++

```
#include <iostream>
#include <list>
using namespace std;
int main() {
    list<string> fila = {"Victor", "Maria", "Jose"};
    while(!fila.empty()) {
        cout << "Despachando a: " << fila.front() << "\n";
        fila.pop_front();
    }
    cout << "No hay mas clientes en fila.\n";
    return 0;
}
```



}

Código en Java:

Java

```
import java.util.LinkedList;
public class Ejercicio10 {
    public static void main(String[] args) {
        LinkedList<String> fila = new LinkedList<>();
        fila.add("Victor"); fila.add("Maria"); fila.add("Jose");
        while(!fila.isEmpty()) {
            System.out.println("Despachando a: " + fila.removeFirst());
        }
        System.out.println("No hay mas clientes en fila.");
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Llenar fila	3 Strings	add() consecutivo	size = 3
2. Ciclo Despacho	!isEmpty() evalúa True	Remueve 1 a 1 de la cabeza	Imprime los 3 nombres
3. Parada	La lista queda vacía	Sale del ciclo while	Muestra "Fila vacía"

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

El uso de colecciones dinámicas como ArrayList y LinkedList en Java (y sus equivalentes vector y list en C++) permite una gestión de memoria mucho más flexible que los arreglos estáticos, previniendo errores críticos de desbordamiento de índices (IndexOutOfBoundsException) al manipular volúmenes variables de información.

Se evidenció que aunque un ArrayList es altamente eficiente para iterar y buscar datos secuencialmente mediante sus índices, la estructura LinkedList destaca por su rendimiento en operaciones estructurales, como agregar o eliminar constantemente elementos en las puntas (cabeza o cola), haciéndola ideal para simular filas o turnos de atención.



La aplicación de los principios de Programación Orientada a Objetos (POO), instanciando objetos con múltiples atributos y guardándolos dentro de listas dinámicas, facilita la construcción de sistemas modulares y escalables, como se demostró en los ejercicios del menú CRUD y el control de inventario.

2.10 Recomendaciones

Emplear estructuras de datos dinámicas por defecto en lugar de arreglos primitivos cuando no se conozca con exactitud la cantidad de datos que ingresará el usuario durante el ciclo de vida de la aplicación.

Implementar siempre bloques de control de excepciones (try-catch) al trabajar con índices de arreglos o listas, para garantizar que el programa mantenga su ejecución y avise al usuario mediante mensajes amigables en caso de una falla técnica.

Estandarizar la nomenclatura de las variables y añadir comentarios breves sobre la lógica de negocio dentro de los ciclos iterativos, ya que esto facilita la legibilidad del código para futuras refactorizaciones o revisiones del docente.

2.11 Referencias bibliográficas

2.12 Anexos

Enlace de GitHub

<https://github.com/VictorChacha1/-APE-06-Arreglos-y-Listas.git>

EVIDENCIAS DE LOS EJERCICIOS

Ejercicio 1

Código:

```
import java.util.ArrayList;
public class Ejercicio1 {
    Run | Debug
    public static void main(String[] args) {
        ArrayList<String> estudiantes = new ArrayList<>();
        estudiantes.add(e: "Victor");
        estudiantes.add(e: "Maria");
        estudiantes.add(e: "Luis");
        estudiantes.add(e: "Ana");
        estudiantes.add(e: "Pedro");
        for(int i = 0; i < estudiantes.size(); i++) {
            System.out.println("Estudiante " + (i+1) + ": " + estudiantes.get(i));
        }
    }
}
```

Funcionamiento:

```
Estudiante 1: Victor
Estudiante 2: Maria
Estudiante 3: Luis
Estudiante 4: Ana
Estudiante 5: Pedro
```



Ejercicio 2

Código:

```
import java.util.ArrayList;
public class Ejercicio2 {
    Run | Debug
    public static void main(String[] args) {
        ArrayList<Double> notas = new ArrayList<>();
        notas.add(e: 9.5); notas.add(e: 8.0); notas.add(e: 7.5);
        notas.add(e: 10.0); notas.add(e: 6.0); notas.add(e: 8.5);
        double suma = 0;
        for(Double n : notas) suma += n;
        System.out.println("Promedio total: " + (suma / notas.size()));
    }
}
```

Funcionamiento:

Promedio total: 8.25

Ejercicio 3

Código:

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio3 {
    Run | Debug
    public static void main(String[] args) {
        ArrayList<String> prods = new ArrayList<>();

        prods.add(e: "Teclado");
        prods.add(e: "Mouse");
        prods.add(e: "Monitor");

        Scanner sc = new Scanner(System.in);

        System.out.print(s: "Buscar producto: ");
        String buscar = sc.next();

        if (prods.contains(buscar)) {
            System.out.println(x: "Producto encontrado");
        } else {
            System.out.println(x: "No existe");
        }

        sc.close();
    }
}
```

Funcionamiento:

Buscar producto: Teclado
Producto encontrado



Ejercicio 4

Código:

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio4 {
    Run | Debug
    public static void main(String[] args) {
        ArrayList<String> lista = new ArrayList<>();
        Scanner sc = new Scanner(System.in);
        int op;

        do {
            System.out.print(s: "1.Add 2.Ver 3.Update 4.Delete 5.Salir: ");
            op = sc.nextInt();

            if (op == 1) {
                lista.add(sc.next());
            } else if (op == 2) {
                for (int i = 0; i < lista.size(); i++) {
                    System.out.println(i + ":" + lista.get(i));
                }
            } else if (op == 3) {
                int id = sc.nextInt();
                if (id < lista.size()) {
                    lista.set(id, sc.next());
                }
            } else if (op == 4) {
                int id = sc.nextInt();
                if (id < lista.size()) {
                    lista.remove(id);
                }
            }
        } while (op != 5);

        sc.close();
    }
}
```

Funcionamiento:

```
1.Add 2.Ver 3.Update 4.Delete 5.Salir: 1
teclado
1.Add 2.Ver 3.Update 4.Delete 5.Salir: 2
0:teclado
1.Add 2.Ver 3.Update 4.Delete 5.Salir: 1
mouse
1.Add 2.Ver 3.Update 4.Delete 5.Salir: 2
0:teclado
1:mouse
1.Add 2.Ver 3.Update 4.Delete 5.Salir: 1
laptop
1.Add 2.Ver 3.Update 4.Delete 5.Salir: 4
11
1.Add 2.Ver 3.Update 4.Delete 5.Salir: 2
0:teclado
1:mouse
2:laptop
1.Add 2.Ver 3.Update 4.Delete 5.Salir: 4
1
1.Add 2.Ver 3.Update 4.Delete 5.Salir: 2
0:teclado
1:laptop
```




Ejercicio 5

Código:

```
import java.util.ArrayList;
class Alumno {
    String nombre; double nota;
    Alumno(String n, double p) { nombre = n; nota = p; }
}
public class Ejercicio5 {
    Run | Debug
    public static void main(String[] args) {
        ArrayList<Alumno> clase = new ArrayList<>();
        clase.add(new Alumno(n: "Victor", p: 9.5));
        clase.add(new Alumno(n: "Maria", p: 8.0));
        for(Alumno a : clase) System.out.println(a.nombre + " - " + a.nota);
    }
}
```

Funcionamiento:

```
Victor - 9.5
Maria - 8.0
```

Ejercicio 6

Código:

```
import java.util.ArrayList;
class Prod {
    String nom; int cant; double pre;
    Prod(String n, int c, double p) { nom=n; cant=c; pre=p; }
}
public class Ejercicio6 {
    Run | Debug
    public static void main(String[] args) {
        ArrayList<Prod> inv = new ArrayList<>();
        inv.add(new Prod(n: "Mouse", c: 10, p: 15.5));
        double total = 0;
        for(Prod p : inv) total += (p.cant * p.pre);
        System.out.println("Valor inventario: $" + total);
    }
}
```

Funcionamiento:

```
Valor inventario: $155.0
```



Ejercicio 7

Código:

```
import java.util.ArrayList;
public class Ejercicio7 {
    Run | Debug
    public static void main(String[] args) {
        int[] arr = new int[3];
        arr[0] = 1; arr[1] = 2; arr[2] = 3;
        System.out.println("Arreglo fijo length: " + arr.length);

        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(e: 1); lista.add(e: 2);
        System.out.println("ArrayList dinamico size: " + lista.size());
    }
}
```

Funcionamiento:

```
Arreglo fijo length: 3
ArrayList dinamico size: 2
```

Ejercicio 8

Código:

```
import java.util.ArrayList;
public class Ejercicio8 {
    Run | Debug
    public static void main(String[] args) {
        int[] arr = new int[2];
        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(e: 10); lista.add(e: 20); lista.add(e: 30); // Funciona perfecto
        System.out.println("ArrayList crece, size: " + lista.size());
        try {
            arr[2] = 30; // Falla, index maximo es 1
        } catch (Exception e) {
            System.out.println(x: "Error en arreglo: Limite superado");
        }
    }
}
```

Funcionamiento:

```
ArrayList crece, size: 3
Error en arreglo: Limite superado
```



Ejercicio 9

Código:

```
import java.util.LinkedList;
public class Ejercicio9 {
    Run | Debug
    public static void main(String[] args) {
        LinkedList<String> turnos = new LinkedList<>();
        turnos.addLast(e: "Turno A");
        turnos.addLast(e: "Turno B");
        System.out.println("Atendiendo: " + turnos.removeFirst());
        System.out.println("Siguiente en espera: " + turnos.getFirst());
    }
}
```

Funcionamiento:

```
Atendiendo: Turno A
Siguiente en espera: Turno B
```

Ejercicio 10

Código:

```
import java.util.LinkedList;
public class Ejercicio10 {
    Run | Debug
    public static void main(String[] args) {
        LinkedList<String> fila = new LinkedList<>();
        fila.add(e: "Victor"); fila.add(e: "Maria"); fila.add(e: "Jose");
        while(!fila.isEmpty()) {
            System.out.println("Despachando a: " + fila.removeFirst());
        }
        System.out.println(x: "No hay mas clientes en fila.");
    }
}
```

Funcionamiento:

```
Despachando a: Victor
Despachando a: Maria
Despachando a: Jose
No hay mas clientes en fila.
```